Lecture 2: Engineering Thinking

Dr. Changlai Du

Learning goals

After this lecture, you should be able to formulate an engineering problem before committing to a solution, define observable success criteria, reason about constraints and trade-offs, choose a minimum viable product, decompose a system into responsibilities, and define abstraction boundaries through clear interface promises.

Contents

1 Engineering Thinking and the Campus Event Challenge 3

1.1 The problem behind the proposed app 3

2 Success Criteria, Constraints, and Trade-offs 4

2.1 What counts as success? 4

2.2 Constraints shape the solution 4

3 A Minimum Viable Product 5

4 Decomposing the Registration System 6

4.1 Inputs and outputs 6

4.2 How the parts complete one registration 6

5 Abstraction and the Boundary Between Parts 8

5.1 A precise interface promise 8

6 Questions Reserved for the Next Lecture 9

6.1 Modules and reuse 9

6.2 A shared development process 9

6.3 A double-booking failure 9

6.4 Debugging with evidence 10

6.5 Testing, validation, and iteration 10

6.6 Synthesis 11

7 Optional Transfer Questions 12

Homework 13

Sources Used for These Notes 14

1 Engineering Thinking and the Campus Event Challenge

Engineering thinking is a disciplined way to frame problems, reason about possible solutions under constraints, and improve those solutions using evidence. Programming is part of this work, but writing code begins only after we understand what the system is meant to accomplish.

The running case study is a campus event-registration service:

> The university wants an app for students to register for campus events. Your team has one week. Where would you begin?

The statement already suggests an app, but it does not yet state the underlying problem. Beginning with screens, buttons, or a database risks building a technically functional system that does not address the real need.

1.1 The problem behind the proposed app

Before selecting a solution, ask:

- Who experiences the problem?
- What happens today?
- Which assumptions should we check with students and organizers?
- Could a simpler process solve the problem without a new app?

Students may currently register through group messages or forms while organizers update spreadsheets by hand. Information can become inconsistent: a student may not know whether a place is confirmed, and an organizer may not have an accurate attendance list. A useful working problem statement is therefore:

Working problem statement

Students cannot reliably determine whether they have a confirmed place, and organizers struggle to maintain an accurate attendee list.

This wording identifies affected users and observable difficulties. “Build an app” names a possible solution, not the problem itself. For a small number of events, a shared form, a spreadsheet, and a clear confirmation procedure might be enough. For repeated events with many participants, a dedicated system may be justified. Evidence from the users and the current process should guide that choice.

Each simpler alternative has a cost. A spreadsheet can record registrations, but the organizer must spend considerable time checking submissions, updating rows, resolving duplicates, and keeping the attendee list consistent. A group chat on a third-party instant-messaging service such as WeChat supports communication, but it may not provide the campus identity information required by the registration process. The university’s existing campus app already includes a login or user-authentication system, so it can associate an action with a verified campus identity. A third-party messaging account does not necessarily establish who the user is within the university. This difference matters when eligibility and registration rules depend on a student’s institutional identity.

Important assumptions include who may register, whether guests are allowed, whether cancellation is possible, how places are confirmed, and whether students actually want another app. Unchecked assumptions often become expensive design mistakes.

2 Success Criteria, Constraints, and Trade-offs

2.1 What counts as success?

A running program does not by itself establish that a project is finished. The program might implement the wrong policy, lose information, exceed event capacity, or give users an ambiguous result. Success must be stated as an observable outcome for the people who depend on the system.

For the event-registration problem, candidate outcomes include:

- a student can find an event and request a place;
- the student can clearly tell whether the request was confirmed;
- the organizer can obtain an accurate attendee list;
- the system never confirms more participants than the event capacity;
- a student's confirmed reservation remains available after logging out and returning.

These outcomes can become acceptance criteria. For example:

Example acceptance criteria

1. When one place remains and one eligible student registers, the system records exactly one reservation and displays confirmation.
2. When an event is full, another registration request receives a clear rejection and does not increase the stored attendee count.

An acceptance criterion connects a requirement to evidence. We can inspect stored information, run a controlled test, and observe whether a student understands the displayed result.

2.2 Constraints shape the solution

The team has one week, four members, limited resources, and limited experience. These constraints make trade-offs unavoidable. Optional features may have to wait, familiar tools may be safer than unfamiliar ones, and the team should protect the correctness of the central registration journey before expanding the product.

One major trade-off concerns user authentication. Implementing a secure login system is complicated. If the university already provides a suitable sign-in service, the team can reuse it rather than rebuild authentication. Reuse does not eliminate responsibility: the team must still check that the component identifies the right users, protects information, and fits the rest of the design.

Trade-offs should not sacrifice the system's essential promise. A small product may omit recommendations or calendar integration, but it must not knowingly overbook events or claim that an unrecorded reservation succeeded.

3 A Minimum Viable Product

Minimum viable product

A minimum viable product (MVP) is the smallest usable version that addresses an important user need and lets the team test a meaningful assumption with real users.

For this case, a complete minimal journey is more useful than several disconnected screens. The first version should allow a student to:

1. sign in through an available identity service;
2. view a list of events and an event's details;
3. request registration;
4. receive a clear success or rejection message;
5. return later and see the registration status.

The organizer should be able to view the resulting attendee list. Behind the interface, the system must remember registrations and enforce the applicable rules, including eligibility and event capacity.

Features that can wait include localization, categories and filters, bookmarking events of interest, social sharing, recommendations, calendar integration, reminders, and an automated waitlist. These ideas may be valuable, but they are not necessary to test whether students can register and understand the outcome.

Letting students try the MVP provides evidence about the design. We can observe whether the registration button is easy to find, whether the confirmation is understandable, and whether a user who returns later can find the recorded status. Feedback can also reveal missing rules or assumptions. A successful database update is not enough if the user cannot tell what happened.

4 Decomposing the Registration System

Decomposition

Decomposition divides a problem into smaller responsibilities whose inputs, outputs, and connections can be explained clearly.

The classroom discussion identified three main parts:

1. the front end, which displays events, accepts user actions, and shows results;
2. the registration logic, which applies the registration rules;
3. storage, which remembers events, users, and reservations.

Authentication supports the journey by supplying a user identity, but it can be an existing university service rather than a newly implemented subsystem.

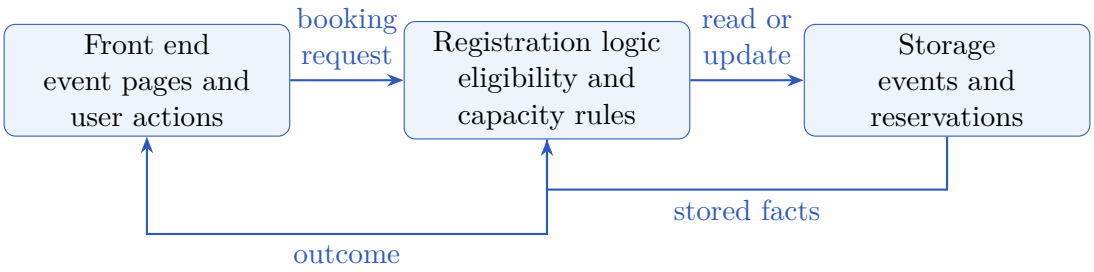


Figure 1: A responsibility-level decomposition of the event-registration system.

4.1 Inputs and outputs

The front end receives user input and event information. A registration action produces a request containing at least the user's identity and the event's identity. The front end later displays the outcome returned by the registration logic.

The registration logic receives the request and the facts needed to evaluate it. It checks rules such as:

- whether the user is eligible for the event;
- whether guests are allowed;
- whether registration is still open;
- whether the event has remaining capacity;
- whether the user already has a reservation.

If the request is allowed, the logic sends an authorized update to storage. Storage accepts queries and updates, remembers the reservation, and later returns stored facts. The logic then reports a confirmed reservation, a rule-based rejection, or a system error.

4.2 How the parts complete one registration

One possible interaction is:

1. The student selects an event and clicks the registration button.
2. The front end sends the student identity and event identity to the registration logic.
3. The registration logic obtains the relevant event and reservation state.
4. The logic applies the rules.
5. If the request is permitted, storage records the reservation.
6. Only after successful storage does the system return confirmation for the front end to display.

The three parts can each appear to work independently while the whole system fails. For example, the logic may produce correct information in a format that storage does not accept. The page may interpret “request received” as “reservation confirmed.” The components therefore need agreed interfaces and shared meanings, not merely individually correct code.

5 Abstraction and the Boundary Between Parts

Abstraction

Abstraction exposes the behavior that another part needs while hiding implementation details that the caller does not need.

The event page does not need to know how reservations are stored. The storage might use a relational database, a spreadsheet, or a text file. The page also does not need to know the database table layout or the internal steps used to check capacity. It needs a clear way to request a reservation and interpret the result.

The same separation works in the other direction. The back end does not need to know whether the registration button appears near the top or bottom of the page, whether ten or twenty events appear at once, or which colors the page uses to distinguish an interesting event from a confirmed registration.

5.1 A precise interface promise

Contract for *reserve a place*

Given a valid student identity and event identity, the operation returns *confirmed* only when the registration satisfies the rules and a durable reservation has been stored. Otherwise, it returns a clear rejection or error without claiming that registration succeeded.

The word *confirmed* has a shared meaning across the boundary. It is not merely an acknowledgement that a click or request reached the server. It means that the rules passed and the system remembers the reservation.

With this contract, the team can change the internal storage method without changing the event page, provided the interface and promised behavior remain the same. The abstraction makes independent change possible, but it does not excuse incorrect implementation. Every implementation behind the boundary must still keep the promise.

6 Questions Reserved for the Next Lecture

The class did not reach the remaining material in the slide deck. The following pages therefore preserve the questions only. No speaker-note answers are included.

6.1 Modules and reuse

Build or reuse?

Which parts should we build ourselves, and which could we reuse?

Modules and reuse

- If the booking policy changes, which part should change?
- Could an existing sign-in component meet our needs?
- What must we check before depending on a reused component?

6.2 A shared development process

Team development

How can four people turn these parts into one reliable app?

Shared process

- How do we know which version to use?
- What should a teammate check before accepting a change?
- What does “done” mean for a feature and for the whole app?

6.3 A double-booking failure

Competing requests

What if two students try to book the last place at the same time?

One place, two requests

A checks availability. One place remains.
B checks availability. One place remains.
A receives confirmation.
B receives confirmation.
Each request followed the usual steps. Why did the system fail?

A symptom and a broken rule

- What rule must always remain true?
- Which observation shows that the rule has failed?
- What do we know so far, and what are we only guessing?

6.4 Debugging with evidence

Finding the cause

How would you find the cause of the double-booking bug?

Evidence from the investigation

One request at a time works. Two overlapping requests can fail. The logs show that both requests read "1 place left" before either saves.

- What hypothesis fits this evidence?
- What experiment would test it?

A fix we can explain

- What is the smallest change that addresses the cause?
- How would we check that the fix works and does not break other behavior?
- Which test should we keep so the bug does not return?

6.5 Testing, validation, and iteration

Readiness

What evidence would convince you that the app is ready for real users?

Correct behavior and useful outcomes

- Which failure would you test first?
- Could all our tests pass while students still find the app confusing?
- How do testing and debugging help us in different ways?

The next version

What should we change after students try the first version?

Feedback and responsibility

- What did the feedback reveal about our original assumptions?
- If AI proposes the change, what must we still check?
- How would we know that the new version is better?

6.6 Synthesis

Engineering thinking

Which kinds of thinking did we use besides writing code?

Your explanation

- How would you explain engineering thinking to a friend?
- Which question will you ask before your next team project?

7 Optional Transfer Questions

These extension questions also appeared after the lecture material and were not discussed in class.

Campus equipment loan service

How would you apply the same thinking to a campus equipment loan service?

A short engineering proposal

- State the problem and one constraint.
- Choose a useful first version.
- Identify the responsibilities and one component you might reuse.
- Propose a failure test.
- Explain how you would investigate a failure.

Homework

Read the assigned material on *Version Control*. Review both the material from Lecture 1 and the engineering-thinking questions from this lecture in preparation for the next quiz.

Sources Used for These Notes

These notes synthesize the slide deck *02 Engineering Thinking.pptx*, its embedded speaker notes for the portion presented in class, and the classroom transcript dated 13 October 2026. The transcript begins after the opening portion because screen recording was not started at the beginning. Slides 1–6 were therefore reconstructed from the deck. The transcript and deck jointly support the detailed notes for slides 7–12. For slides 13 onward, only the questions shown on the slides have been reproduced; answers from the speaker notes have intentionally been omitted.